

| Information and Communication Technology

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY
BIG DATA ANALYTICS LABORATORY
(303105362)

6TH SEMESTER

COMPUTER SCIENCE & ENGINEERING
DEPARTMENT

LABORATORY MANUAL

CERTIFICATE

This is to certify that PANKAJ YADAV with enrolment no.
2303031250098 has successfully completed his/her
laboratory experiments in the **BIG DATA ANALYTICS**
LABORATORY (303105362) from the Department of
COMPUTER SCIENCE AND ENGINEERING during the
academic year 2025-26.

Information and Communication Technology

INDEX

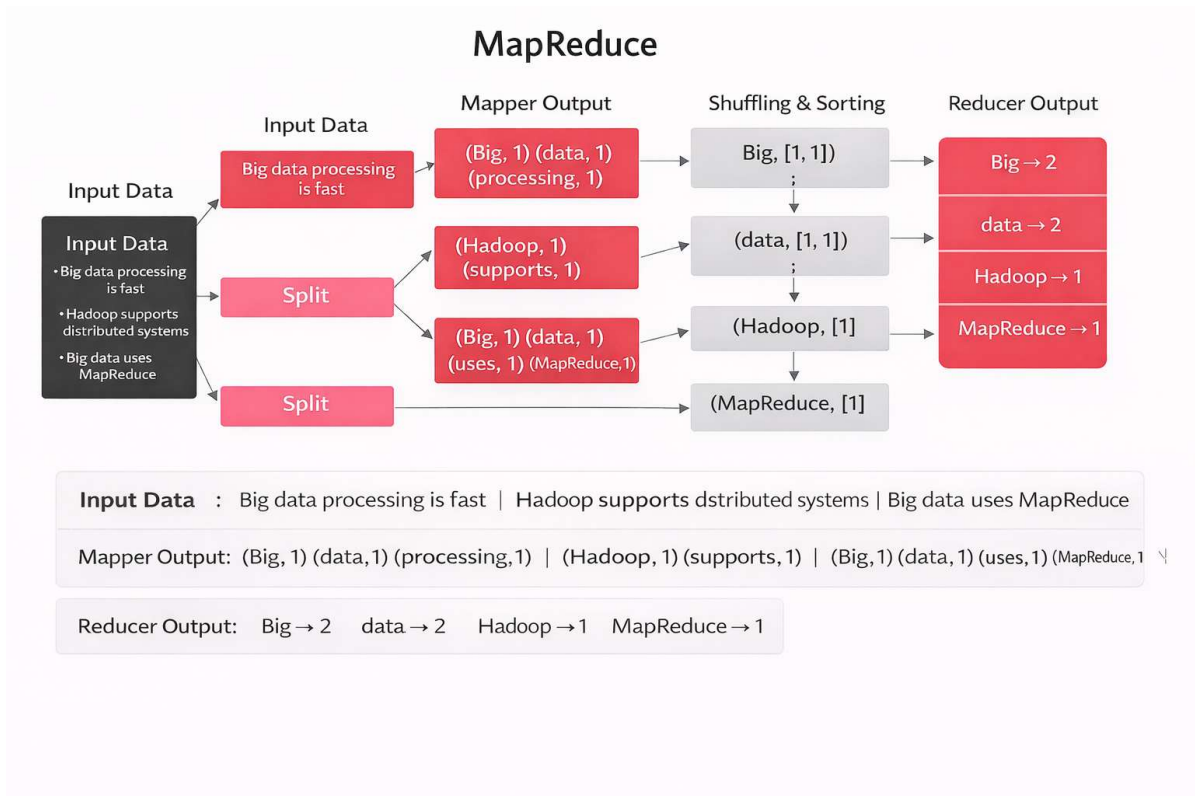
SR NO	NAME	START DATE	END DATE	MARKS	SIGN
1	To understand the overall programming architecture using Map Reduce API.				
2	Write a program of Word Count in Map Reduce over HDFS.				
3	Basic CRUD operations in MongoDB.				
4	Store the basic information about students such as roll no, name, date of birth, and address of student using various collection types such as List, Set and Map.				
5	Basic commands available for the Hadoop Distributed File System.				
6	Basic commands available for HIVE Query Language.				
7	Basic commands of HBASE Shell.				
8	Creating the HDFS tables and loading them in Hive and learn joining of tables in Hive.				

Practical -1

Aim: To Understand the overall programming Architecture using MapReduce.

- Hadoop mainly relies on two core components, **MapReduce and HDFS**, which together provide high performance and reliability.
- MapReduce is a **distributed programming framework** designed to process very large datasets by executing tasks in parallel across multiple systems.
- In this model, input data is **divided into smaller chunks**, processed independently, and then **merged** to generate the final output.
- MapReduce libraries are available in **multiple programming languages**, each offering different performance optimizations.
- The main objective of MapReduce is to **break down complex jobs into smaller tasks** and combine their results, thereby minimizing network load and reducing overall processing overhead in the cluster.
- The MapReduce workflow is primarily divided into **two stages: the Map stage and the Reduce stage**.
 1. **map(), filter(), and reduce()** are functional programming tools available in Python that follow the MapReduce concept.
 2. These functions are commonly implemented using **lambda expressions** for concise and efficient operations.

Information and Communication Technology



- **map():**

It is used to perform the same operation on all elements of a collection such as a list, tuple, or set and returns the updated values.

```

https://colab.research.google.com/#scrollTo=IT-Eod_cdqNr

Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all Copy to Drive

# map() example

items = [15, 21, 33, 64, 95]

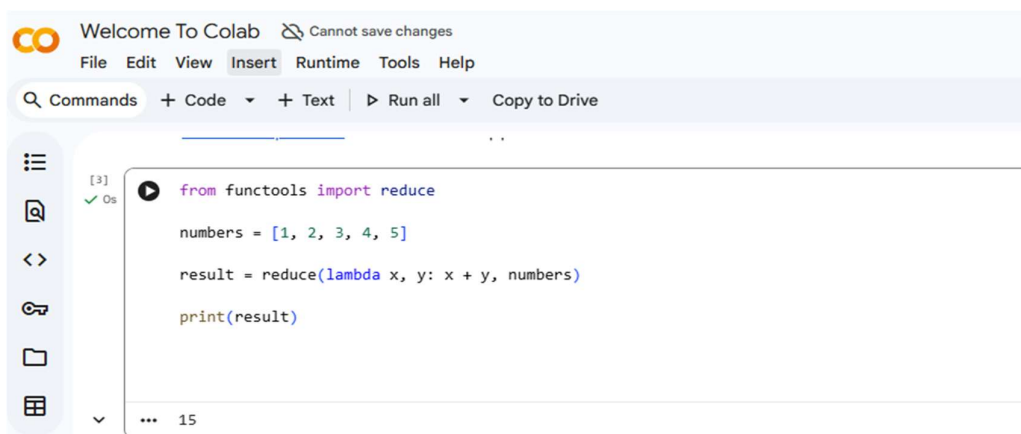
a = list(map(lambda x: x**3, items))

print(a)

... [3375, 9261, 35937, 262144, 857375]
```

Information and Communication Technology

- `reduce()` : The `reduce()` function processes all elements of an iterable by repeatedly applying a given function and combines them into **one final output value**. It reduces a sequence of values into a single result such as a sum, product, or maximum.



The screenshot shows a Google Colab interface. At the top, it says "Welcome To Colab" and "Cannot save changes". Below that is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar contains "Commands". Below the menu bar are buttons for "+ Code", "+ Text", "Run all", and "Copy to Drive". On the left sidebar, there are icons for file management. The main code area shows a code cell with the following Python code:

```
[3] ✓ Os
from functools import reduce

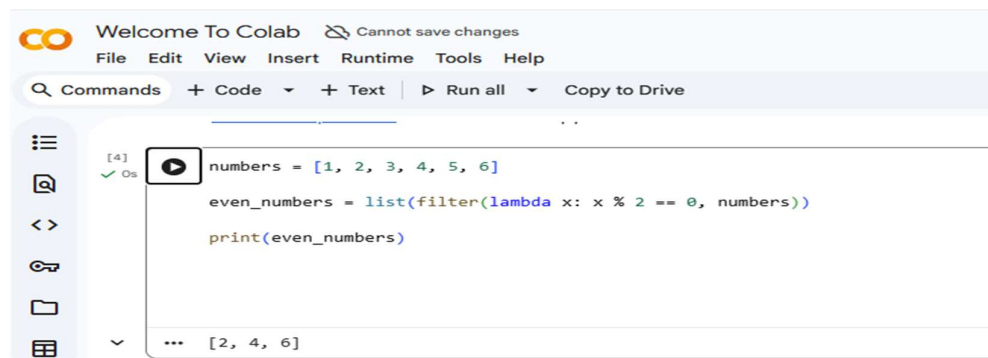
numbers = [1, 2, 3, 4, 5]

result = reduce(lambda x, y: x + y, numbers)

print(result)
```

The output of the code cell is "15".

- `filter()` : The `filter()` function in Python evaluates each element of an iterable using a given condition and returns only those elements for which the condition is **True**.



The screenshot shows a Google Colab interface. At the top, it says "Welcome To Colab" and "Cannot save changes". Below that is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar contains "Commands". Below the menu bar are buttons for "+ Code", "+ Text", "Run all", and "Copy to Drive". On the left sidebar, there are icons for file management. The main code area shows a code cell with the following Python code:

```
[4] ✓ Os
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers)
```

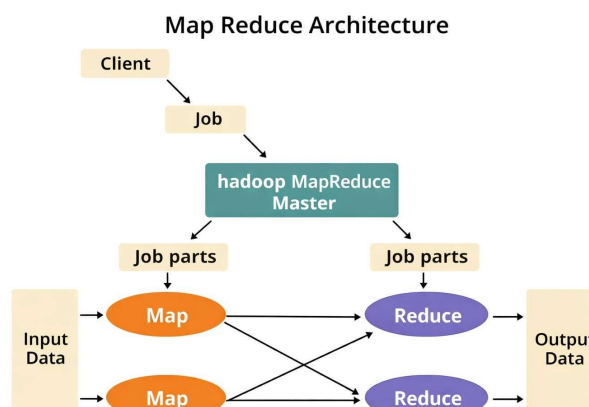
The output of the code cell is "[2, 4, 6]".

Practical: 2

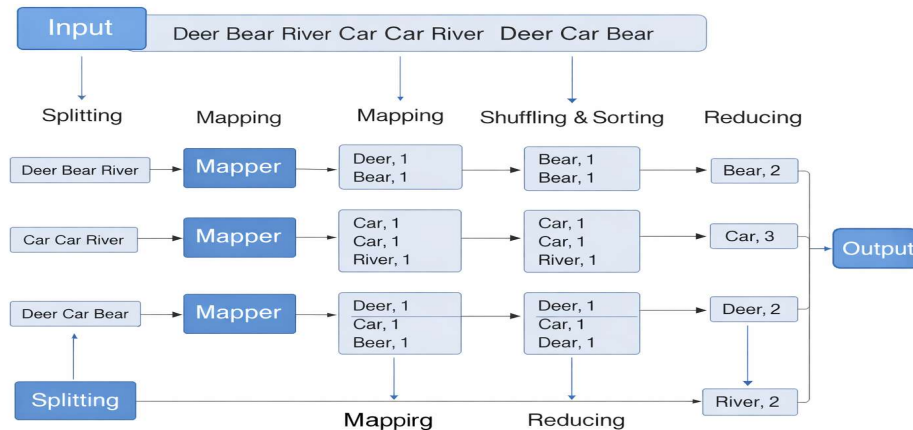
- **Aim: Write a program of Word Count in Map Reduce over HDFS.**

Description: MapReduce is a distributed computing model designed to handle very large volumes of data by dividing the work across many interconnected computers called nodes. These nodes together form a cluster, where data is commonly stored in a distributed storage system such as HDFS. Instead of moving data to a central processor, the computation is moved to the location where the data already exists, improving efficiency and performance.

In MapReduce, the original input data is divided into smaller, independent chunks so that multiple tasks can run simultaneously. Each chunk is transformed into key–value pairs, which are then processed by the Mapper function. The Mapper applies user-defined logic to each input pair and generates intermediate key–value pairs. These intermediate pairs may differ in type and structure from the original input.



The output produced by the Mapper is forwarded to the Reducer phase. Before reduction, the system automatically groups and sorts the intermediate key–value pairs based on their keys. The Reducer function then processes each group, applies aggregation or summarization logic, and generates the final output. The completed results are stored back into HDFS in the required format.



MapReduce is a programming model designed for processing large amounts of data in parallel across a distributed cluster of computers. Here, I'll provide a basic example of a word count program using MapReduce in Hadoop.

Environment Preparation

Before executing the program, ensure the following setup is complete:

1. **Hadoop Installation**

Hadoop should be properly installed and configured on your system.

2. **HDFS Configuration**

The Hadoop Distributed File System (HDFS) must be running, and input text files should be uploaded to HDFS.

Word Count Using MapReduce

1. Mapper Component

The Mapper is responsible for processing the input data. It reads the input text line by line, separates each line into individual words, and outputs each word paired with the value 1.

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
```



```
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WCMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    private static final IntWritable countOne = new IntWritable(1);
    private Text outputWord = new Text();

    @Override
    protected void map(LongWritable inputKey, Text inputValue, Context context)
        throws IOException, InterruptedException {

        String record = inputValue.toString();
        String[] tokenizedWords = record.split("\\s+");

        for (String token : tokenizedWords) {
            outputWord.set(token);
            context.write(outputWord, countOne);
        }
    }
}
```

2.Reducer Component

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WCReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text inputKey, Iterable<IntWritable> inputValues, Context context)
        throws IOException, InterruptedException {
```

```
int totalCount = 0;

for (IntWritable val : inputValues) {
    totalCount += val.get();
}

context.write(inputKey, new IntWritable(totalCount));
}
}
```

3.Driver Component

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WCDriver {

    public static void main(String[] args) throws Exception {

        Configuration configuration = new Configuration();
        Job job = Job.getInstance(configuration, "Hadoop Word Count");

        job.setJarByClass(WCDriver.class);
        job.setMapperClass(WCMapper.class);
        job.setCombinerClass(WCReducer.class);
        job.setReducerClass(WCReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
}
```

Executing the MapReduce Application

After writing the Mapper, Reducer, and Driver classes, the next step is to compile the code, store the input data in HDFS, and execute the MapReduce job on the Hadoop cluster.

Step 1: Compile Code and Create JAR File

First, compile all Java source files and generate class files using the Hadoop classpath. Then package the compiled classes into a JAR file.

```
javac -classpath $(hadoop classpath) -d wc_classes \  
WCMapper.java WCReducer.java WCDriver.java
```

Create the JAR file:

```
jar -cf wc_app.jar -C wc_classes/ .
```

Step 2: Upload Input Data to HDFS

Create a directory in HDFS to store the input files and upload the text file.

```
hdfs dfs -mkdir /wc_input  
hdfs dfs -put input.txt /wc_input
```

Step 3: Execute the MapReduce Job

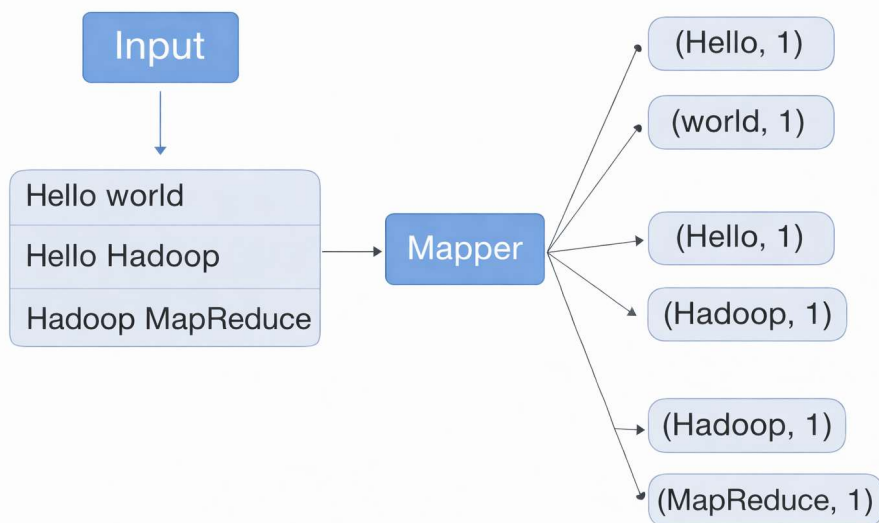
Run the MapReduce application by specifying the JAR file, Driver class, input directory, and output directory.

```
hadoop jar wc_app.jar WCDriver /wc_input /wc_output
```

Step 4: View the Output Results

Once the job finishes successfully, display the output file generated by the Reducer.

```
hdfs dfs -cat /wc_output/part-r-00000
```



CODE for program without Mapreduce:

```
import urllib.request

story = 'http://sixty-north.com/c/t.txt'
request = urllib.request.Request(story)
response = urllib.request.urlopen(request)

each_word = []
same_words = {}

# Collect all words
for line in response:
    line_words = line.split()
    for word in line_words:
```

```
each_word.append(word)
```

```
# Count words
```

```
for words in each_word:
```

```
    if words.lower() not in same_words:
```

```
        same_words[words.lower()] = 1
```

```
    else:
```

```
        same_words[words.lower()] += 1
```

```
# Print output
```

```
for each in same_words.keys():
```

```
    print("word = ", each, ", count = ", same_words[each])
```

```
word = b'it' , count = 10
word = b'was' , count = 11
word = b'the' , count = 14
word = b'best' , count = 1
word = b'of' , count = 12
word = b'times' , count = 2
word = b'worst' , count = 1
word = b'age' , count = 2
word = b'wisdom' , count = 1
word = b'foolishness' , count = 1
```

Practical -3

Aim: Basic CRUD Operations in MongoDB.

Program -1:

```
db.createCollection("staff")

db.staff.insertMany([
  { empld: 101, name: "Rahul", dept: "IT" },
  { empld: 102, name: "Sneha", dept: "HR" },
  { empld: 103, name: "Amit", dept: "IT" }
])

db.staff.find({ dept: "IT" })

db.staff.insertOne({ empld: 104, name: "Neha", dept: "Finance" })

db.staff.updateOne(
  { name: "Neha" },
  { $set: { name: "Neha Sharma" } }
)

db.staff.find({ dept: "Finance" })

db.staff.deleteOne({ dept: "HR" })
```


Program -2 Create orders collection and items to the database.

```
db.createCollection("orders")

db.orders.insertMany( [

  { id: 0, name: "Pepperoni", size: "small", price: 19,
    quantity: 10, date: ISODate( "2021-03-13T08:14:30Z" ) },

  { id: 1, name: "Pepperoni", size: "medium", price: 20,
    quantity: 20, date : ISODate( "2021-03-13T09:13:24Z" ) },

  { id: 2, name: "Pepperoni", size: "large", price: 21,
    quantity: 30, date : ISODate( "2021-03-17T09:22:12Z" ) },

  { id: 3, name: "Cheese", size: "small", price: 12,
    quantity: 15, date : ISODate( "2021-03-13T11:21:39.736Z" ) },

  { id: 4, name: "Cheese", size: "medium", price: 13,
    quantity: 50, date : ISODate( "2022-01-12T21:23:13.331Z" ) },

  { id: 5, name: "Cheese", size: "large", price: 14,
    quantity: 10, date : ISODate( "2022-01-12T05:08:13Z" ) },

  { id: 6, name: "Vegan", size: "small", price: 17,
    quantity: 10, date : ISODate( "2021-01-13T05:08:13Z" ) },

  { id: 7, name: "Vegan", size: "medium", price: 18,
    quantity: 10, date : ISODate( "2021-01-13T05:10:13Z" ) }

] )

db.orders.find({size: "medium"});

db.orders.insert({id: 9, name: "Vegan", size: "medium", price: 8,
  quantity: 5, date : ISODate( "2021-01-22T05:10:13Z" ) })
```



```
db.orders.updateMany({name:'Vegan'},{$set:{name:'Veg'}}
db.orders.find({name: 'Veg'});
db.orders.remove({name:'Pepperoni'})
db.orders.find({ $and: [ {name: 'Veg'}, { size: "small"} ] })

db.orders.find()
```

Insert, Update, Delete (CRUD Operations)

1. Staff Collection View



- **READ:** Showing all staff records in the "staff" collection

2. Update Reema to Meha



- **UPDATE:** Changing "Reema → Meha".
Result: 1 document updated

3. Delete Amit Employees



- **DELETE:** Removing all employees in "Amit" department.
Result: 2 documents deleted

Order Queries (Find Operations)

1. Find "Finance" Employees



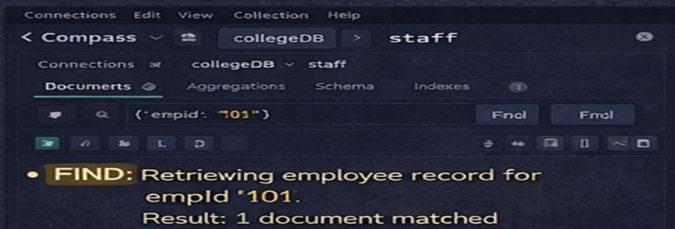
- **FIND:** Retrieving all employees in "Finance" department.
Result: 1 document matched

2. Find "Meha" (Previously Reema)



- **FIND:** Retrieving record where name is "Mehha".
Result: 1 document matched

3. Find Empld 101



- **FIND:** Retrieving employee record for empld '101'.
Result: 1 document matched

PRACTICAL – 4

Aim: Store the basic information about students such as roll no, name, Date of birth, and address of student using various collection types such as List, Set and Hadoop.

Code:

- Create database studentdb and Make Collection With name "student" and Follow Queries

✓ Created Database

> use studentdb

switched to DB studentdb

✓ Create Collection

> db.createCollection("student")

✓ Insert Records Into student Collection using **insertOne()** method

```
> db.student.insertOne({no:1, name:"ANUJ MULHAR",dob: "2004-01-30",e_mail: "thefunvlogs0@gmail.com", phone:"9904857092",address: {"B": "118","Nr.Kosamdi": "KUMKUM BUNGALOWS VALIA ROAD ANKLESHWAR","zipcode": "393001"},Branch:"CSE", marks:[58,96,55]})
```

Output:

```
db.student.insertOne({...})  
  
{  
  "acknowledged" : true,  
  "insertedId" : ObjectId("65a1b2c3d4e5f67890123456")  
}
```

Insert Records Into student Collection using **insertMany()** method.

```
db.student.insertMany([  
  {no:2, name:"KUNAL NAKUM",dob: "2005-09-26",e_mail: "kunalnakum.123@gmail.com", phone:"9313564747",address:  
    {"Kalam Bhawan-c ": "401","street": "WAGHODIA","zipcode":  
      "391760"},Branch:"CSE", marks:[30,55,100]},  
  {no:3, name:"Aditya patel",dob: "2002-10-21",e_mail: "adityapatel123@gmail.com",  
    phone:"9313564747",address: {"Kalam Bhawan -C": "101","street":  
      "Limda","zipcode": "391760"},Branch:"CSE", marks:[60,50,42]})  
])
```

Output:

```
db.student.insertMany([...])  
  
{  
  "acknowledged" : true,  
  "insertedIds" : {  
    "0" : ObjectId("65a1b2c3d4e5f67890123457"),  
    "1" : ObjectId("65a1b2c3d4e5f67890123458")  
  }  
}
```

List out all Document

```
>db.student.find()
```

```
db.student.find()
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123456"),  
  "no" : 1,  
  "name" : "ANUJ MULHAR",  
  "dob" : "2004-01-30",  
  "e_mail" : "thefunvlogs0@gmail.com",  
  "phone" : "9904857092",  
  "Branch" : "CSE",  
  "marks" : [58, 96, 55]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123457"),  
  "no" : 2,  
  "name" : "KUNAL NAKUM",  
  "dob" : "2005-09-26",  
  "e_mail" : "kunalnakum.123@gmail.com",  
  "phone" : "9313564747",  
  "Branch" : "CSE",  
  "marks" : [30, 55, 100]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123458"),  
  "no" : 3,  
  "name" : "ADITYA PATEL",  
  "dob" : "2002-10-21",  
  "e_mail" : "adityapatel123@gmail.com",  
  "phone" : "9313564747"
```

update name of no 1 as "kunal sinh"

```
studentdb> db.student.update({no:1},{ $set:{name:"kunal sinh"}}
```

```
db.student.update({no:1}, {$set:{name:"kunal sinh"}})
```

```
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

- ✓ To Find Document From the student collection where name begins with A

```
studentdb> db.student.find({name:/^A/})
```

```
db.student.find({name:/^A/})
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123458"),  
  "no" : 3,  
  "name" : "ADITYA PATEL",  
  "dob" : "2002-10-21",  
  "e_mail" : "adityapatel123@gmail.com",  
  "phone" : "9313564747",  
  "Branch" : "CSE",  
  "marks" : [60, 50, 42]  
}
```

Count:

It is used to count the number of document in the collections

```
db.student.count()
```

```
db.student.count()
```

```
3
```

limit() method is used to specify the maximum number of documents the cursor will return.

```
db.student.find().limit(2);
```

```
db.student.find().limit(2)
```

```
{ "_id" : ObjectId("65a1b2c3d4e5f67890123456"),  
  "no" : 1,  
  "name" : "kunal sinh",  
  "dob" : "2004-01-30",  
  "Branch" : "CSE",  
  "marks" : [58, 96, 55]  
}  
  
{ "_id" : ObjectId("65a1b2c3d4e5f67890123457"),  
  "no" : 2,  
  "name" : "KUNAL NAKUM",  
  "dob" : "2005-09-26",  
  "Branch" : "CSE",  
  "marks" : [30, 55, 100]  
}
```

set

Adds new fields to documents. \$set outputs documents that contain all existing fields from the input documents and newly added fields.

```
db.scores.insertMany([
```

```
{_id: 1, student: "Mulhar", homework: [ 10, 5, 10 ], quiz: [ 10, 8 ], extraCredit:  
0 },
```

```
{_id: 2, student: "ANUJ", homework: [ 5, 6, 5 ], quiz: [ 8, 8 ], extraCredit: 8 }}
```

```
db.scores.insertMany([
  { _id: 1, student: "Mulhar", homework: [10, 5, 10], quiz: [10, 8], extraCredit: 0 },
  { _id: 2, student: "ANUJ", homework: [5, 6, 5], quiz: [8, 8], extraCredit: 8 }
])

{
  "acknowledged" : true,
  "insertedIds" : {
    "0" : 1,
    "1" : 2
  }
}
```

- SET behavior, you should use \$addToSet, which avoids duplicates.

```
db.student.update(
  { no: 1 },
  { $addToSet: { marks: 96 } }
)
```

Output:

```
studentdb> db.student.update(
... { no: 1 },
... { $addToSet: { marks: 96 } }
)
WriteResult({ "nMatched" : 1, "nModified" : 1 })
```


Hadoop

Hadoop is used to store and process large student datasets in distributed manner using HDFS. MongoDB can act as a data source before transferring large data to Hadoop for analytics.

```
hdfs dfs -mkdir /student  
hdfs dfs -put studentdata.txt /student  
hdfs dfs -ls /student
```

Output:

```
$ hdfs dfs -ls /student  
Found 1 items  
-rw-r--r--  1 user supergroup laymra 2048 2024-04-24 10:30 /student/studentdata.txt
```